

1. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.
1. Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int global_var = 10;           // Global variable
static int static_var = 20;    // Static variable
void code_function()
{
    printf("Code address      : %p\n", (void *)code_function);
}
int main()
{
    int stack_var = 30;         // Stack variable
    static int local_static = 40; // Static variable
    int *heap_var = malloc(sizeof(int)); // Heap variable
    if (heap_var == NULL)
    {
        perror("malloc");
        return 1;
    }

    *heap_var = 50;

    printf("==== Process Memory Addresses ==== \n");

    printf("Code address      : %p\n", (void *)code_function);
    printf("Global variable   : %p\n", (void *)&global_var);
    printf("Static variable    : %p\n", (void *)&static_var);
    printf("Local static       : %p\n", (void *)&local_static);
    printf("Heap variable      : %p\n", (void *)heap_var);
    printf("Stack variable     : %p\n", (void *)&stack_var);

    printf("\nProcess ID: %d\n", getpid());

    printf("\n==== /proc/%d/maps ==== \n\n", getpid());

    char command[100];
    sprintf(command, "cat /proc/%d/maps", getpid());
    system(command);

    free(heap_var);

    return 0;
}
```

```

prajwal@PRAJWAL:~/os$ ./p7
==== Process Memory Addresses ====
Code address      : 0x631f3ac1b249
Global variable   : 0x631f3ac1e010
Static variable   : 0x631f3ac1e014
Local static      : 0x631f3ac1e018
Heap variable     : 0x631f78bbe010
Stack variable    : 0x7fff9ea62bc4

Process ID: 864

==== /proc/864/maps ====
631f3ac1a000-631f3ac1b000 r--p 00000000 08:30 77414 /home/prajwal/os/p7
631f3ac1b000-631f3ac1c000 r-xp 00001000 08:30 77414 /home/prajwal/os/p7
631f3ac1c000-631f3ac1d000 r--p 00002000 08:30 77414 /home/prajwal/os/p7
631f3ac1d000-631f3ac1e000 r--p 00002000 08:30 77414 /home/prajwal/os/p7
631f3ac1e000-631f3ac1f000 rw-p 00003000 08:30 77414 /home/prajwal/os/p7
631f78bbe000-631f78bdf000 rw-p 00000000 00:00 0 [heap]
7d8dce400000-7d8dce428000 r--p 00000000 08:30 14176 /usr/lib/x86_64-linux-gnu/libc.so.6
7d8dce428000-7d8dce5c0000 r-xp 00028000 08:30 14176 /usr/lib/x86_64-linux-gnu/libc.so.6
7d8dce5c0000-7d8dce60e000 r--p 001c0000 08:30 14176 /usr/lib/x86_64-linux-gnu/libc.so.6
7d8dce60e000-7d8dce612000 r--p 0020d000 08:30 14176 /usr/lib/x86_64-linux-gnu/libc.so.6
7d8dce612000-7d8dce614000 rw-p 00211000 08:30 14176 /usr/lib/x86_64-linux-gnu/libc.so.6
7d8dce614000-7d8dce621000 rw-p 00000000 00:00 0
7d8dce76a000-7d8dce76d000 rw-p 00000000 00:00 0
7d8dce775000-7d8dce777000 rw-p 00000000 00:00 0
7d8dce777000-7d8dce77b000 r--p 00000000 00:00 0 [vvar]
7d8dce77b000-7d8dce77d000 r--p 00000000 00:00 0 [vvar_vclock]
7d8dce77d000-7d8dce77f000 r-xp 00000000 00:00 0 [vdso]
7d8dce77f000-7d8dce780000 r--p 00000000 08:30 13931 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7d8dce780000-7d8dce7af000 r-xp 00001000 08:30 13931 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7d8dce7af000-7d8dce7ba000 r--p 00030000 08:30 13931 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7d8dce7ba000-7d8dce7bc000 r--p 0003b000 08:30 13931 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7d8dce7bc000-7d8dce7bd000 rw-p 0003d000 08:30 13931 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7d8dce7bd000-7d8dce7be000 rw-p 00000000 00:00 0
7fff9ea4000-7fff9ea65000 rw-p 00000000 00:00 0 [stack]
prajwal@PRAJWAL:~/os$

```

8. Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```

#include <stdio.h>
#include <stdlib.h>
int main()
{
    int *a, *b, *c;
    // malloc
    a = malloc(5 * sizeof(int));

    if (a == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
        a[i] = i + 1;

    printf("malloc: ");
    for (int i = 0; i < 5; i++)
        printf("%d ", a[i]);

    printf("\n");

    // calloc
    b = calloc(5, sizeof(int));

    if (b == NULL)
    {
        printf("calloc failed\n");
        free(a);
        return 1;
    }

    printf("calloc: ");
    for (int i = 0; i < 5; i++)
        printf("%d ", b[i]);

    printf("\n");

    // realloc
    c = realloc(a, 10 * sizeof(int));

    if (c == NULL)
    {
        printf("realloc failed\n");
    }
}

```

```

prajwal@PRAJWAL:~/os$ nano p8.c
prajwal@PRAJWAL:~/os$ gcc p8.c -o p8
prajwal@PRAJWAL:~/os$ ./p8
malloc: 1 2 3 4 5
calloc: 0 0 0 0 0
After realloc: 1 2 3 4 5 6 7 8 9 10
prajwal@PRAJWAL:~/os$ |

```